

BÁO CÁO ĐỒ ÁN CUỐI KÌ
MÔN HỌC: PHÂN TÍCH XỬ LÝ ẢNH
CHƯƠNG TRÌNH HỖ TRỢ CHẤM THI VÀ SỐ HÓA BÀI THI
TRẮC NGHIỆM

Nhóm thực hiện: Nhóm 10
Nguyễn Thế Minh Nhật

Mục lục

1	TỔNG QUAN ĐỀ TÀI	3
1.1	Đặt vấn đề	3
1.2	Mục tiêu đề tài	3
1.3	Phạm vi và Đối tượng nghiên cứu	3
1.3.1	Phạm vi nghiên cứu	3
1.3.2	Đối tượng nghiên cứu	4
1.4	Phương pháp tiếp cận	4
1.5	Cấu trúc báo cáo	4
2	CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG	5
2.1	Tổng quan về Xử lý ảnh số (Digital Image Processing)	5
2.1.1	Ảnh xám (Grayscale)	5
2.1.2	Làm trơn ảnh (Gaussian Blur)	5
2.1.3	Phát hiện biên (Canny Edge Detection)	6
2.1.4	Biến đổi phối cảnh (Perspective Transform)	6
2.1.5	Phân ngưỡng thích nghi (Adaptive Thresholding)	7
2.2	Cơ sở lý thuyết về Nhận dạng quang học (Deep Learning OCR)	8
2.2.1	Mạng phát hiện văn bản (Text Detection): CRAFT	8
2.2.2	Mạng nhận dạng chuỗi (Text Recognition): CRNN + CTC	9
2.2.3	Hậu xử lý dữ liệu (Post-processing Pipeline)	10
2.3	Công nghệ và Thư viện nền tảng	11
2.3.1	OpenCV (Xử lý ma trận ảnh)	11
2.3.2	NumPy (Đại số tuyến tính)	11
2.3.3	ReportLab (Sinh dữ liệu lập trình)	11
2.4	Kiến trúc Phần mềm (Software Architecture)	11
3	PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG	12
3.1	Kiến trúc mã nguồn (System Architecture)	12
3.1.1	Lớp Xử lý Logic (Core Logic Layer)	14
3.1.2	Lớp Trực quan hóa (Visualization Layer)	14
3.1.3	Lớp Điều phối và Cấu hình (Coordination & Configuration)	14
3.2	Thiết kế Quy trình hoạt động (Workflow Design)	14
3.3	Thiết kế Dữ liệu (Data Structures)	15
3.3.1	Cấu trúc Template (coordinates.json)	15
3.3.2	Cấu trúc Đáp án (answer_key.csv)	16
4	CÀI ĐẶT VÀ HIỆN THỰC	17
4.1	Module Sinh phiếu thi tự động (Auto Sheet Generator)	17
4.2	Module Xử lý ảnh tiền kỳ (Preprocessing)	17
4.2.1	Thuật toán phát hiện khung giấy	17
4.2.2	Biến đổi phối cảnh (Perspective Transform)	17
4.3	Module Chấm thi (OMR Engine)	18
4.3.1	Kỹ thuật Phân ngưỡng thích nghi (Adaptive Thresholding)	18
4.3.2	Logic xác định ô tô (Bubble Detection)	18
4.4	Module Nhận dạng chữ (OCR Engine)	19
4.4.1	Quy trình xử lý	19
4.4.2	Hậu xử lý dữ liệu bằng Regex	19

4.5	Tinh chỉnh Mô hình Nhận dạng (OCR Fine-tuning)	19
4.5.1	Chuẩn bị dữ liệu (Data Preparation)	19
4.5.2	Cấu hình Huấn luyện (Training Configuration)	20
4.5.3	Xử lý bất đồng bộ kiến trúc (Transfer Learning Challenge)	20
4.6	Đánh giá Thực nghiệm và Chiến lược Tối ưu hóa (Optimization Strategy)	20
4.6.1	Phân tích hạn chế của các Model OCR	20
4.6.2	Chiến lược Hợp nhất OMR - OCR	21
5	KẾT QUẢ THỰC NGHIỆM	22
5.1	Môi trường và Dữ liệu kiểm thử	22
5.1.1	Thiết lập kịch bản (Test Setup)	22
5.1.2	Tham số cấu hình	22
5.2	Kết quả Huấn luyện Mô hình (Training Results)	22
5.3	Đánh giá Hiệu năng Nhận dạng (Evaluation)	23
5.3.1	Đánh giá hiệu năng OCR (Performance Evaluation)	23
5.3.2	Kết quả chấm thi (OMR Evaluation)	23
5.4	Kết quả Chấm thi (OMR Results)	23
5.4.1	Nhật ký xử lý (Console Log)	23
5.4.2	Trực quan hóa kết quả (Visualization)	24
5.5	Kết quả Nhận dạng thông tin (OCR Results)	25
5.6	Tổng kết Hiệu năng và Hạn chế	26
5.6.1	Hiệu năng đạt được (Performance Summary)	26
5.6.2	Hạn chế tồn tại (Current Limitations)	26
6	KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	27
6.1	Kết luận	27
6.1.1	Kết quả đạt được	27
6.1.2	Bài học kinh nghiệm	27
6.2	Hướng phát triển trong tương lai (Future Work)	27
	Tài liệu tham khảo	29

Chương 1

TỔNG QUAN ĐỀ TÀI

1.1 Đặt vấn đề

Trong lĩnh vực Thị giác máy tính (Computer Vision), bài toán xử lý phiếu thi trắc nghiệm (OMR) là một chủ đề kinh điển, thường được sử dụng để minh họa cho các kỹ thuật xử lý ảnh cơ bản như nhị phân hóa, phát hiện biên và biến đổi hình học.

Tuy nhiên, khi áp dụng vào thực tế với thiết bị phổ thông như camera điện thoại, bài toán này nảy sinh nhiều thách thức kỹ thuật thú vị mà các phương pháp truyền thống khó giải quyết trọn vẹn, ví dụ như:

1. **Chất lượng ảnh đầu vào không đồng đều:** Ảnh chụp thường bị nghiêng, méo, hoặc chịu ảnh hưởng của điều kiện ánh sáng (bóng đổ, lóa sáng) khiến việc phân ngưỡng (thresholding) trở nên khó khăn.
2. **Vấn đề nhận dạng chữ viết tay:** Ngoài việc chấm điểm trắc nghiệm, nhu cầu số hóa các thông tin viết tay (như Số báo danh) đòi hỏi sự kết hợp với các mô hình Học sâu (Deep Learning) thay vì chỉ xử lý ảnh đơn thuần.

Với mong muốn vận dụng các kiến thức đã học trong môn Phân tích Xử lý ảnh vào một bài toán thực tế cụ thể, nhóm quyết định thực hiện đề tài **"SmartGrader"**. Đây là một nỗ lực bước đầu nhằm xây dựng chương trình hỗ trợ chấm thi quy mô nhỏ, đồng thời là cơ hội để nhóm rèn luyện kỹ năng lập trình và tư duy giải quyết vấn đề.

1.2 Mục tiêu đề tài

Mục tiêu cốt lõi của đồ án không dừng lại ở việc tạo ra một phần mềm hoàn chỉnh, mà tập trung vào việc **thử nghiệm và đánh giá hiệu quả** của các giải thuật xử lý ảnh khi áp dụng vào dữ liệu thật. Các mục tiêu cụ thể bao gồm:

- **Tự động hóa việc định nghĩa Template (Auto Generator):** Thay thế phương pháp đo đặc tọa độ thủ công (để sai sót) bằng cách lập trình sinh phiếu thi tự động. Mục tiêu là tạo ra bộ dữ liệu mẫu (Ground Truth) có độ tin cậy cao để phục vụ quá trình chấm.
- **Cải thiện thuật toán OMR:** Tìm hiểu và so sánh hiệu quả của kỹ thuật Phân ngưỡng thích nghi (Adaptive Thresholding) so với phân ngưỡng toàn cục trong việc xử lý các ảnh bị nhiễu hoặc ánh sáng không đều [?].
- **Thử nghiệm công nghệ OCR:** Bước đầu tìm hiểu cách tích hợp thư viện EasyOCR để nhận dạng các trường thông tin viết tay (Số báo danh), kết hợp với các bộ lọc dữ liệu (Regex) để đánh giá khả năng "làm sạch" kết quả của mô hình [?].
- **Xây dựng quy trình xử lý (Pipeline):** Tổ chức mã nguồn thành một luồng xử lý khép kín: từ ảnh đầu vào → tiền xử lý → nhận dạng → xuất kết quả, giúp rèn luyện tư duy thiết kế hệ thống.

1.3 Phạm vi và Đối tượng nghiên cứu

1.3.1 Phạm vi nghiên cứu

Do giới hạn về thời gian và nguồn lực, đề tài tập trung giải quyết các bài toán cụ thể sau:

- **Dữ liệu đầu vào:** Chương trình chỉ xử lý các tệp tin hình ảnh (.jpg, .png) hoặc tài liệu quét (.pdf) của phiếu trả lời trắc nghiệm do chính chương trình sinh ra (theo Template định sẵn).
- **Công nghệ sử dụng:** Mã nguồn được viết bằng ngôn ngữ **Python**, kết hợp các thư viện mã nguồn mở phổ biến: **OpenCV** (cho xử lý ảnh), **EasyOCR** (cho nhận dạng ký tự), và **ReportLab** (để tạo phiếu thi mẫu).
- **Giới hạn kỹ thuật:** Chương trình yêu cầu ảnh đầu vào phải nhìn thấy rõ 4 góc định vị (neo) của phiếu thi để thực hiện phép biến đổi hình học (Warp Perspective). Đề tài chưa xử lý các trường hợp phiếu thi bị rách, mất góc hoặc biến dạng quá lớn [7].

1.3.2 Đối tượng nghiên cứu

Đồ án tập trung tìm hiểu và vận dụng các đối tượng kiến thức sau:

- **Các thuật toán xử lý ảnh cơ bản:** Phát hiện biên (Canny Edge Detection), Tìm đường bao (Contour Retrieval) và Biến đổi phối cảnh (Perspective Transform).
- **Kỹ thuật nhị phân hóa:** So sánh hiệu quả giữa Otsu's Binarization và Adaptive Gaussian Thresholding trong điều kiện ánh sáng thực tế.
- **Công nghệ OCR:** Tìm hiểu cấu trúc và cách fine-tuning mô hình EasyOCR để nhận dạng chữ số viết tay.

1.4 Phương pháp tiếp cận

Để mã nguồn dễ dàng quản lý và bảo trì, nhóm áp dụng tư duy tổ chức mã nguồn theo hướng **Module hóa (Modular Design)**, chia tách các thành phần xử lý riêng biệt:

1. **Module Xử lý chính (Core Processing):** Bao gồm `omr_engine.py` (thực hiện thuật toán chấm điểm) và `ocr_engine.py` (gọi thư viện nhận dạng ký tự). Các tham số vị trí được tách riêng ra file cấu hình `coordinates.json` thay vì viết cứng trong code (Hard-coding).
2. **Module Hiển thị (Visualization):** File `renderer.py` chịu trách nhiệm vẽ lại kết quả chấm (vòng tròn xanh/đỏ) lên ảnh gốc để người dùng dễ quan sát.
3. **Module Điều phối (Main Controller):** File `main.py` đóng vai trò kết nối, đọc dữ liệu ảnh đầu vào, gọi lần lượt các hàm xử lý và xuất kết quả cuối cùng ra màn hình hoặc file Excel.

1.5 Cấu trúc báo cáo

Báo cáo được trình bày trong 6 chương với nội dung cụ thể như sau:

- **Chương 1: Tổng quan đề tài.** Trình bày lý do chọn đề tài, mục tiêu, phạm vi và phương pháp tiếp cận của nhóm.
- **Chương 2: Cơ sở lý thuyết.** Tóm tắt các kiến thức nền tảng về Xử lý ảnh (Grayscale, Edge Detection, Warping) và công nghệ OCR được sử dụng trong đồ án.
- **Chương 3: Phân tích và Thiết kế hệ thống.** Mô tả luồng dữ liệu (Flowchart) và cách tổ chức thư mục, cấu trúc dữ liệu đầu vào/đầu ra.
- **Chương 4: Cài đặt và Hiện thực.** Đi sâu vào chi tiết cài đặt các thuật toán quan trọng: Sinh đề tự động, Chống nhiễu bằng Adaptive Threshold, và Hậu xử lý dữ liệu OCR.
- **Chương 5: Kết quả thực nghiệm.** Trình bày các kịch bản kiểm thử trên ảnh chụp thực tế và đánh giá sơ bộ độ chính xác của chương trình.
- **Chương 6: Kết luận và Hướng phát triển.** Tổng kết các kiến thức đã học được, nhìn nhận các hạn chế còn tồn tại và đề xuất hướng cải tiến trong tương lai.

Chương 2

CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

2.1 Tổng quan về Xử lý ảnh số (Digital Image Processing)

Trong khuôn khổ đồ án, các kỹ thuật xử lý ảnh đóng vai trò tiền xử lý (Preprocessing) nhằm chuẩn hóa dữ liệu đầu vào. Mục tiêu là biến đổi ảnh chụp thô (raw image) thành dạng ảnh chuẩn mà máy tính có thể "hiểu" được. Các kỹ thuật chính được áp dụng bao gồm:

2.1.1 Ảnh xám (Grayscale)

Ảnh xám là ảnh mà mỗi điểm ảnh (pixel) chỉ mang thông tin về cường độ sáng (Luminance), loại bỏ hoàn toàn thông tin màu sắc (Chrominance).

Việc chuyển đổi từ không gian màu RGB sang Grayscale là bước bắt buộc vì hai lý do:

- **Giảm độ phức tạp tính toán:** Giảm dữ liệu từ 3 kênh màu xuống 1 kênh, giúp tăng tốc độ xử lý lên gấp 3 lần.
- **Tăng hiệu quả phát hiện biên:** Các thuật toán tìm biên hoạt động dựa trên sự thay đổi cường độ sáng, việc giữ lại màu sắc thường gây nhiễu không cần thiết.

Trong mã nguồn (`src/utlis/image_utils.py`), nhóm sử dụng công thức chuyển đổi tiêu chuẩn dựa trên độ nhạy của mắt người:

$$Y = 0.299 \times R + 0.587 \times G + 0.114 \times B \quad (2.1)$$

Hàm sử dụng: `cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)`.

2.1.2 Làm trơn ảnh (Gaussian Blur)

Tại sao cần làm trơn? Thuật toán phát hiện biên Canny (bước sau) hoạt động dựa trên đạo hàm (tính tốc độ thay đổi màu sắc). Đạo hàm cực kỳ nhạy cảm với nhiễu. Một điểm nhiễu nhỏ (ví dụ: vết bụi) cũng tạo ra sự thay đổi màu đột ngột, khiến Canny nhận nhầm đó là một cạnh. Do đó, việc làm mờ để xóa nhiễu là bắt buộc.

Cơ chế hoạt động: Đồ án sử dụng bộ lọc Gaussian thay vì bộ lọc trung bình (Mean Filter) thông thường.

- **Mean Filter:** Lấy trung bình cộng tất cả các điểm lân cận. Nhược điểm là làm mờ cả các cạnh quan trọng.
- **Gaussian Filter:** Tính trung bình có trọng số (Weighted Average). Các điểm ảnh ở gần tâm sẽ có trọng số lớn, các điểm ở xa có trọng số nhỏ dần theo hình quả chuông (Bell Curve).

Ma trận hạt nhân (Kernel): Trong mã nguồn, hàm `cv2.GaussianBlur` sử dụng kernel kích thước 5×5 . Về mặt toán học, đây là phép tích chập (convolution) ảnh gốc với một ma trận xấp xỉ Gaussian, ví dụ:

$$K = \frac{1}{273} \begin{bmatrix} 1 & 4 & 7 & 4 & 1 \\ 4 & 16 & 26 & 16 & 4 \\ 7 & 26 & 41 & 26 & 7 \\ 4 & 16 & 26 & 16 & 4 \\ 1 & 4 & 7 & 4 & 1 \end{bmatrix} \quad (2.2)$$

Nhìn vào ma trận trên, ta thấy điểm trung tâm (giá trị 41) đóng góp nhiều nhất vào kết quả, các điểm rìa (giá trị 1) đóng góp rất ít.

Kết quả: Phép toán này giúp loại bỏ nhiễu hạt tần số cao (high-frequency noise) nhưng vẫn giữ lại được độ sắc nét tương đối của các đường biên chính, tạo điều kiện tốt nhất cho thuật toán Canny.

2.1.3 Phát hiện biên (Canny Edge Detection)

Thuật toán Canny được coi là chuẩn mực trong phát hiện biên nhờ khả năng tối ưu hóa giữa việc phát hiện đầy đủ các cạnh và hạn chế tối đa việc bắt nhầm nhiễu. Quy trình toán học gồm 4 bước:

Bước 1: Khử nhiễu (Noise Reduction) Sử dụng bộ lọc Gaussian (như đã trình bày ở mục 2.1.2) để làm trơn ảnh, vì các toán tử đạo hàm rất nhạy cảm với nhiễu.

Bước 2: Tính Gradient cường độ (Intensity Gradient) Sau khi làm trơn, thuật toán sử dụng bộ lọc Sobel theo hai hướng ngang (G_x) và dọc (G_y) để tính đạo hàm bậc nhất của ảnh.

Cường độ biên (Edge Gradient) G và hướng biên θ tại mỗi điểm ảnh (x, y) được tính như sau:

$$G(x, y) = \sqrt{G_x^2 + G_y^2} \quad (2.3)$$

$$\theta(x, y) = \arctan\left(\frac{G_y}{G_x}\right) \quad (2.4)$$

Hướng biên θ rất quan trọng vì nó sẽ quyết định hướng quét cho bước tiếp theo.

Bước 3: Triệt tiêu phi cực đại (Non-maximum Suppression) Mục tiêu của bước này là "làm mảnh" (thinning) đường biên. Kết quả của bước 2 thường tạo ra các đường biên dày và mờ.

Thuật toán sẽ duyệt qua toàn bộ các điểm ảnh. Tại mỗi điểm, nó so sánh cường độ G của điểm đó với hai điểm lân cận theo hướng vuông góc với biên θ .

- Nếu điểm hiện tại là lớn nhất (cực đại cục bộ): Giữ lại.
- Nếu không: Gán giá trị về 0 (loại bỏ).

Kết quả là ta thu được các đường biên mảnh (độ rộng 1 pixel).

Bước 4: Phân ngưỡng trễ (Hysteresis Thresholding) Đây là bước quyết định giúp Canny vượt trội hơn các thuật toán khác trong việc xử lý biên đứt đoạn. Thuật toán phân loại các điểm ảnh thành 3 nhóm dựa trên hai ngưỡng T_{min} và T_{max} :

- **Biên mạnh (Strong Edge):** Cường độ $G > T_{max}$. Điểm này chắc chắn là biên.
- **Biên yếu (Weak Edge):** Cường độ $T_{min} \leq G \leq T_{max}$. Điểm này nằm ở vùng tranh chấp.
- **Không phải biên (Non-Edge):** Cường độ $G < T_{min}$. Loại bỏ ngay lập tức.

Cơ chế liên thông: Một điểm "Biên yếu" sẽ được giữ lại (chuyển thành biên mạnh) **nếu và chỉ nếu** nó có kết nối (nằm cạnh) với một điểm "Biên mạnh". Ngược lại, nó sẽ bị coi là nhiễu và bị loại bỏ. Cơ chế này giúp nối liền các nét đứt quãng do ánh sáng yếu mà không làm tăng nhiễu.

2.1.4 Biến đổi phối cảnh (Perspective Transform)

Do góc chụp nghiêng, các đường song song trong thực tế (như cạnh của phiếu thi) sẽ hội tụ tại một điểm ảo trong ảnh, tạo ra hiện tượng méo hình thang. Để khôi phục góc nhìn trực diện (Top-down view), hệ thống sử dụng phép biến đổi phối cảnh dựa trên **Ma trận Homography**.

Cơ sở toán học: Quá trình này tìm một ma trận M kích thước 3×3 để ánh xạ điểm nguồn (x, y) sang điểm đích (u, v) . Vì đây là không gian tọa độ đồng nhất (Homogeneous Coordinates), phương trình được biểu diễn như sau:

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} \times \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \quad (2.5)$$

Tọa độ thực tế của điểm ảnh mới (u, v) được tính bằng cách chuẩn hóa (chia cho tham số tỉ lệ w'):

$$u = \frac{x'}{w'} = \frac{h_{11}x + h_{12}y + h_{13}}{h_{31}x + h_{32}y + h_{33}}, \quad v = \frac{y'}{w'} = \frac{h_{21}x + h_{22}y + h_{23}}{h_{31}x + h_{32}y + h_{33}} \quad (2.6)$$

Do ma trận M có 8 bậc tự do (thường chuẩn hóa $h_{33} = 1$), hệ thống cần tối thiểu **4 cặp điểm tương ứng** (4 góc phiếu thi) để giải hệ phương trình tìm 8 ẩn số còn lại.

Cơ chế nội suy (Interpolation): Vì tọa độ (u, v) tính được từ phép biến đổi ma trận thường là số thực (ví dụ: 10.6, 20.4), không trùng khớp với lưới pixel số nguyên của ảnh kỹ thuật số, hàm `cv2.warpPerspective` mặc định sử dụng kỹ thuật **Nội suy song tuyến tính (Bilinear Interpolation)**.

Ví dụ minh họa: Giả sử sau khi biến đổi, ta cần tìm giá trị màu tại điểm thực $P(10.6, 20.4)$. Điểm này nằm giữa 4 pixel nguyên:

- Góc trên-trái $A(10, 20)$
- Góc trên-phải $B(11, 20)$
- Góc dưới-trái $C(10, 21)$
- Góc dưới-phải $D(11, 21)$

Thay vì chỉ lấy giá trị của điểm gần nhất (Nearest Neighbor) gây ra hiện tượng răng cưa, thuật toán sẽ tính toán sự đóng góp màu sắc dựa trên khoảng cách:

- Điểm P lệch về phía $x = 11$ (0.6 đơn vị) và $y = 20$ (0.4 đơn vị).
- Do đó, màu của P sẽ chịu ảnh hưởng nhiều nhất từ điểm $B(11, 20)$ và ít nhất từ điểm $C(10, 21)$.

Công thức tổng quát:

$$f(x, y) \approx \sum_{i=0}^1 \sum_{j=0}^1 f(x_i, y_j) \cdot w_{ij} \quad (2.7)$$

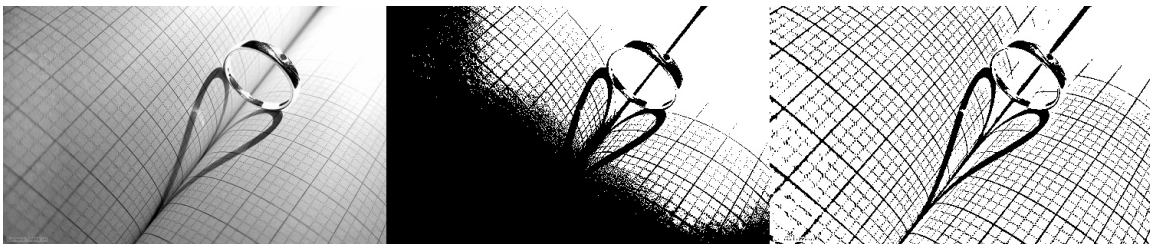
Trong đó w_{ij} là diện tích hình chữ nhật đối diện, đảm bảo tổng trọng số bằng 1. Kỹ thuật này giúp ảnh đầu ra (output) giữ được độ mượt mà của các đường nét, đặc biệt quan trọng khi phóng to hoặc xoay ảnh.

2.1.5 Phân ngưỡng thích nghi (Adaptive Thresholding)

Hạn chế của Phân ngưỡng toàn cục (Global Thresholding): Các phương pháp truyền thống như Otsu tìm một giá trị ngưỡng T duy nhất cho toàn bộ bức ảnh. Giả sử $T = 127$:

- Mọi điểm ảnh $< 127 \rightarrow$ Đen (Mực).
- Mọi điểm ảnh $> 127 \rightarrow$ Trắng (Giấy).

Tuy nhiên, trong thực tế khi ảnh bị bóng đổ, cường độ sáng của vùng giấy trắng trong bóng râm có thể tụt xuống mức 100. Lúc này, Otsu sẽ nhận nhầm toàn bộ vùng bóng râm đó là "Mực đen", dẫn đến kết quả sai lệch hoàn toàn.



Hình 2.1: So sánh kết quả phân ngưỡng trên ảnh thực tế có bóng đổ: (1) Ảnh gốc đầu vào; (2) Kết quả thất bại khi dùng Global Threshold (Otsu); (3) Kết quả xử lý thành công bằng Adaptive Gaussian Thresholding.

Giải pháp Adaptive Gaussian Thresholding: Để khắc phục, nhóm sử dụng phương pháp phân ngưỡng thích nghi. Ngưỡng $T(x, y)$ tại mỗi điểm ảnh không cố định mà được tính toán động dựa trên các điểm lân cận:

$$T(x, y) = \mu_{local}(x, y) - C \quad (2.8)$$

Trong đó:

- μ_{local} : Là trung bình có trọng số Gaussian của vùng lân cận (Block Size 51×51).
- C : Hằng số trừ đi (Constant), giúp tinh chỉnh độ nhạy (được thiết lập $C = 10$).

Ví dụ minh họa cơ chế hoạt động: Xét một điểm ảnh P nằm trong vùng bóng râm (Shadow Region):

- Giá trị thực của điểm P (giấy trắng bị tối): $I(P) = 110$.
- Trung bình độ sáng của vùng lân cận P : $\mu_{local} = 115$ (do cả vùng đều tối).

So sánh cách xử lý:

- Nếu dùng **Global** ($T = 127$): $110 < 127 \rightarrow$ Kết luận là **ĐEN** (Sai, vì thực tế đây là giấy trắng).
- Nếu dùng **Adaptive** ($C = 10$):

$$T_{local} = 115 - 10 = 105$$

$$I(P) = 110 > 105 \rightarrow \text{Kết luận là } \mathbf{TRẮNG} \text{ (Đúng)}.$$

Kết luận: Nhờ cơ chế "nước nổi bè nổi" (ngưỡng thay đổi theo nền), Adaptive Thresholding tách biệt hiệu quả vết mực in ngay cả khi cường độ sáng của nền giấy thay đổi liên tục trên toàn bức ảnh.

2.2 Cơ sở lý thuyết về Nhận dạng quang học (Deep Learning OCR)

Để giải quyết bài toán số hóa thông tin thí sinh (Tên, Số báo danh), đề án sử dụng thư viện EasyOCR. Khác với các công cụ OCR truyền thống (như Tesseract) dựa trên các đặc trưng thủ công, EasyOCR là một hệ thống End-to-End Deep Learning, kết hợp hai mạng nơ-ron chuyên biệt cho hai tác vụ: Phát hiện (Detection) và Nhận dạng (Recognition).

2.2.1 Mạng phát hiện văn bản (Text Detection): CRAFT

Để định vị vị trí các dòng chữ trong ảnh, EasyOCR sử dụng kiến trúc **CRAFT (Character Region Awareness for Text Detection)**.

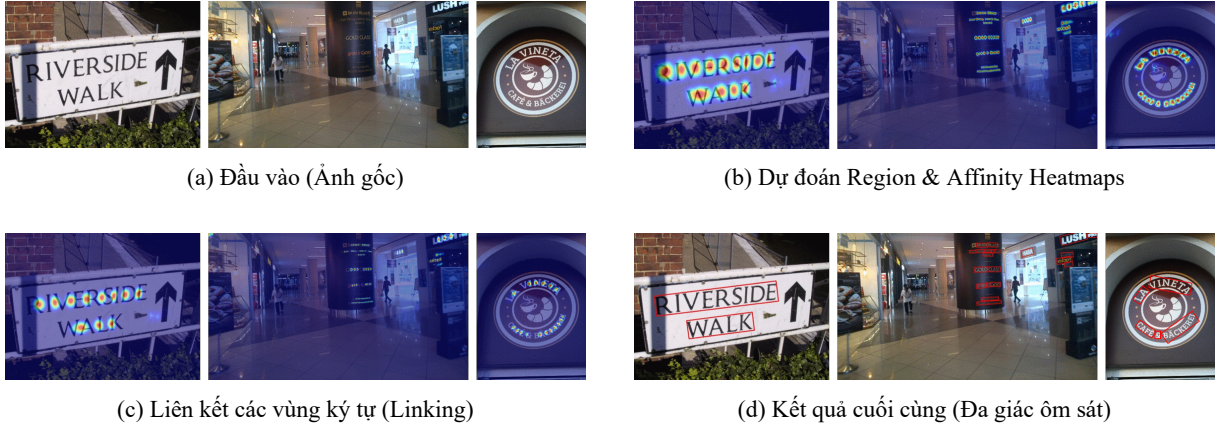
Sự khác biệt về cách tiếp cận: Hầu hết các mô hình phát hiện vật thể (như YOLO, SSD) tiếp cận theo hướng "Top-down" (dự đoán khung bao cho toàn bộ từ). Ngược lại, CRAFT tiếp cận theo hướng "**Bottom-up**" (**Từ dưới lên**). Nó phát hiện từng ký tự riêng lẻ trước, sau đó mới liên kết chúng lại thành từ. Điều này giúp mô hình cực kỳ linh hoạt với các dạng văn bản có hình dạng bất kỳ.

Cơ chế Heatmap (Bản đồ nhiệt): Mô hình không trả về tọa độ hộp ngay lập tức mà trả về hai bản đồ xác suất (Probability Maps) cho từng pixel:

- **Region Score:** Xác suất pixel đó là tâm của một ký tự.
- **Affinity Score:** Xác suất pixel đó là khoảng không gian nối giữa hai ký tự liền kề.

Quá trình hậu xử lý (Post-processing): Sau khi có được hai bản đồ nhiệt này, hệ thống thực hiện các bước:

1. **Nhị phân hóa:** Áp dụng ngưỡng (Thresholding) để lọc bỏ các vùng có xác suất thấp.
2. **Liên kết (Linking):** Các vùng Region sẽ được nối lại với nhau nếu vùng Affinity giữa chúng có giá trị cao.
3. **Tạo đa giác (Polygon Generation):** Thay vì vẽ hình chữ nhật cứng nhắc, CRAFT tạo ra một đường bao đa giác ôm sát theo chiều cong của dòng chữ.



Hình 2.2: Minh họa quy trình xử lý "Bottom-up" của mô hình CRAFT (trích xuất từ mô phỏng thực tế): Từ ảnh đầu vào (a), mô hình sinh ra các bản đồ nhiệt xác suất (b), sau đó liên kết các vùng ký tự dựa trên điểm Affinity (c) để tạo ra đường bao đa giác cuối cùng ôm sát dòng chữ (d).

Ưu điểm trong đồ án: Nhờ cơ chế liên kết mềm dẻo này, CRAFT xử lý rất tốt các trường hợp phiếu thi bị cong vênh khi chụp, điều mà các mô hình Bounding Box truyền thống thường bị lỗi (cắt lẹm mất chữ hoặc lấy thừa nền).

2.2.2 Mạng nhận dạng chuỗi (Text Recognition): CRNN + CTC

Sau khi định vị được vùng văn bản (Text Region), bài toán chuyển sang giai đoạn nhận dạng chuỗi (Sequence Recognition). EasyOCR áp dụng kiến trúc **CRNN** (đề xuất bởi Shi et al.), bao gồm 3 thành phần chính hoạt động như một đường ống (pipeline) liên mạch:

1. Trích xuất đặc trưng (Feature Extraction - CNN)

Đầu vào là ảnh cắt (cropped image) của từ vựng đã được chuẩn hóa kích thước (thường là cao 32 pixel).

- Nhóm sử dụng mạng nền (Backbone) là **ResNet** để trích xuất các bản đồ đặc trưng (Feature Maps).
- Cơ chế Map-to-Sequence:** Đây là bước quan trọng để chuyển đổi dữ liệu từ không gian hình ảnh (2D) sang không gian chuỗi (1D). Bản đồ đặc trưng đầu ra của CNN sẽ được chia cắt theo chiều dọc (column-wise) thành các lát cắt (slices). Mỗi lát cắt này đóng vai trò là một "từ mã" (frame) đại diện cho một vùng nhỏ của ảnh theo thứ tự từ trái sang phải.

2. Mô hình hóa chuỗi (Sequence Modeling - RNN)

Chuỗi các đặc trưng hình ảnh được đưa vào mạng **Bi-LSTM (Bidirectional Long Short-Term Memory)**.

- Tại sao cần 2 chiều?** Chữ viết có tính ngữ cảnh mạnh. Ví dụ trong ảnh mờ, nếu chỉ nhìn từ trái sang, chữ 'H' có thể giống chữ 'N'. Nhưng nếu biết ký tự sau nó là 'o' và 't' (trong từ 'Hot'), mạng sẽ tự sửa lỗi dự đoán. Bi-LSTM quét dữ liệu theo cả hai hướng (Quá khứ → Tương lai và ngược lại) để tối ưu hóa khả năng này.
- Đầu ra của tầng này là xác suất dự đoán của từng ký tự tại mỗi lát cắt thời gian.

3. Giải mã (Decoding - CTC Loss)

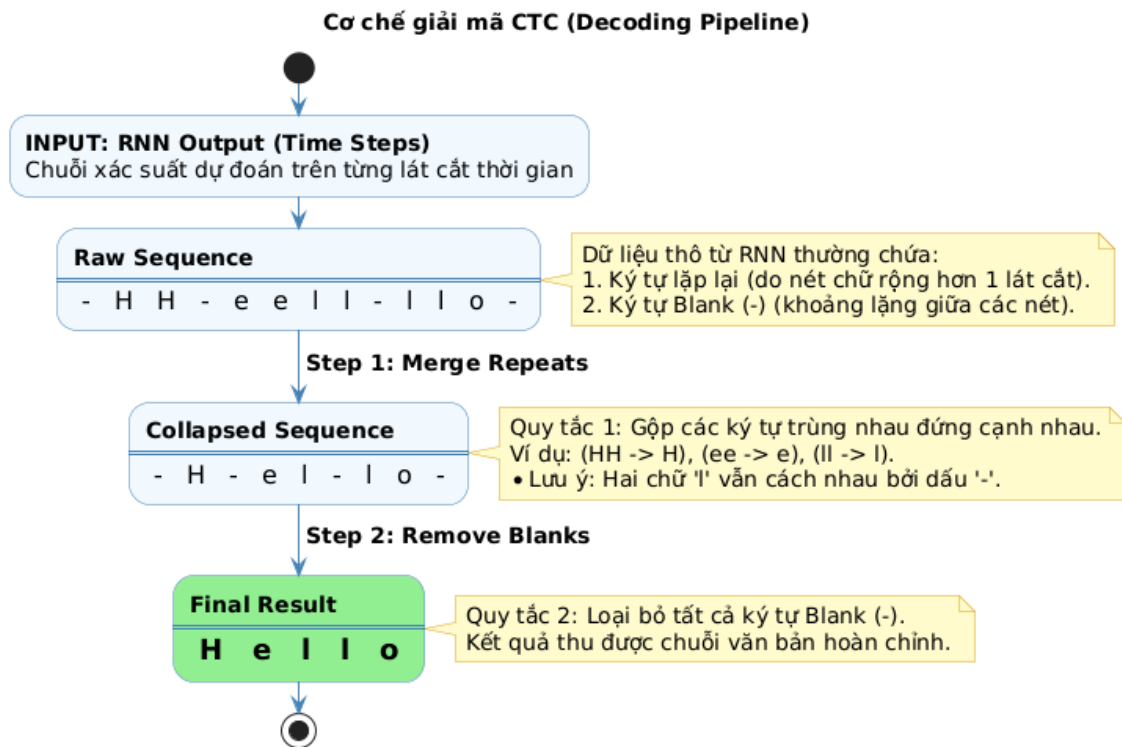
Đây là lớp cuối cùng giải quyết vấn đề "Bất đồng bộ" (Alignment Problem). Vì chiều rộng của ảnh (số lượng lát cắt) luôn lớn hơn số lượng ký tự thực tế (ví dụ: ảnh chữ "AI" có thể bị cắt thành 10 lát), RNN sẽ sinh ra rất nhiều ký tự trùng lặp.

Thuật toán **CTC (Connectionist Temporal Classification)** giới thiệu một ký tự đặc biệt gọi là **"Blank token"** (ký hiệu: ") để giải quyết vấn đề này qua 2 quy tắc:

- Gộp ký tự trùng:** Các ký tự giống nhau đứng liền kề sẽ bị gộp thành 1 (VD: $ll \rightarrow l$).
- Xóa Blank:** Ký tự Blank dùng để phân cách các ký tự trùng lặp thực sự (VD: từ "Hello" có 2 chữ 'l', cần có blank ở giữa $l - l$ để không bị gộp).

Ví dụ quá trình giải mã:

- Đầu ra RNN (Raw): -H-He--l-l-l--l-o-
- Gộp trùng lặp: -H-He-l-l-l-o-
- Loại bỏ Blank: Hello



Hình 2.3: Minh họa cơ chế giải mã CTC: Từ chuỗi dự đoán thô chứa nhiều ký tự lặp và ký tự trống (-), thuật toán tinh gọn lại thành văn bản có nghĩa cuối cùng.

2.2.3 Hậu xử lý dữ liệu (Post-processing Pipeline)

Mô hình Deep Learning hoạt động dựa trên xác suất, do đó đầu ra thường chứa nhiễu (Stochastic Noise). Để đảm bảo tính toàn vẹn dữ liệu (Data Integrity), hệ thống áp dụng cơ chế hậu xử lý 2 bước:

1. Sửa lỗi nhầm lẫn (Homoglyph Correction)

OCR thường xuyên nhận diện sai các ký tự có hình dáng tương đồng. Trước khi lọc, hệ thống sử dụng từ điển ánh xạ (Mapping Dictionary) để khôi phục giá trị đúng dựa trên ngữ cảnh của trường thông tin:

- **Trường Số (SBD):** Ánh xạ các ký tự giống số về số.
Quy tắc: '0', 'o', 'D', 'Q' → '0'; 'I', 'l', 'b' → '1'; 'S', 's' → '5'.
- **Trường Chữ (Họ tên):** Ngược lại, ánh xạ các số về ký tự (ví dụ: '0' → 'O').

2. Lọc định dạng (Regex Filtering)

Sau bước sửa lỗi, Biểu thức chính quy (Regular Expression) được sử dụng như một bộ lọc tất định (Deterministic Filter) cuối cùng:

- **Trường SBD:** Pattern `\D+` (xóa sạch các ký tự phi số còn sót lại).
- **Trường Họ tên:** Pattern `[^a-zA-Z\s\p{L}]` (loại bỏ ký tự đặc biệt, giữ lại chữ cái bao gồm cả Tiếng Việt Unicode).

Bảng 2.1: Ví dụ thực tế về hiệu quả của quy trình hậu xử lý

Trường	Đầu ra OCR (Raw)	Xử lý (Logic)	Kết quả (Clean)
SBD	211103SS	Thay O→0, S→5	21110355
	21.11.03.55	Regex xóa dấu chấm	21110355
Họ Tên	Nguyễn Văn A@	Regex xóa ký tự lạ	Nguyễn Văn A
	Trần 0anh	Thay 0→O	Trần 0anh

2.3 Công nghệ và Thư viện nền tảng

2.3.1 OpenCV (Xử lý ma trận ảnh)

OpenCV không chỉ là công cụ đọc ảnh, mà là thư viện hiện thực hóa các thuật toán Toán học thị giác máy tính. Trong đồ án, OpenCV chịu trách nhiệm thực hiện các phép biến đổi không gian (Spatial Transformations) và phân tích cấu trúc tô pô (Topological Structure Analysis) thông qua việc tìm đường bao (Contours).

2.3.2 NumPy (Đại số tuyến tính)

Trong khoa học máy tính, một bức ảnh kỹ thuật số thực chất là một ma trận đa chiều (Tensor).

- Ảnh xám là ma trận 2D ($H \times W$).
- Ảnh màu là ma trận 3D ($H \times W \times 3$).

Thư viện NumPy cung cấp các cấu trúc dữ liệu hiệu năng cao để thực hiện các phép toán đại số tuyến tính (nhân ma trận, trượt cửa sổ) với tốc độ gần như C/C++, đóng vai trò nền tảng cho mọi tính toán pixel trong đồ án.

2.3.3 ReportLab (Sinh dữ liệu lập trình)

Thay vì tiếp cận theo hướng WYSIWYG (như Word), ReportLab cho phép "vẽ" file PDF thông qua các lệnh lập trình.

Ý nghĩa khoa học của việc này nằm ở tính chính xác tọa độ (Coordinate Precision). Mọi vị trí ô tròn, dòng kẻ đều được định nghĩa bằng đơn vị điểm (points) chính xác tuyệt đối, tạo ra bộ dữ liệu tham chiếu (Ground Truth) hoàn hảo để kiểm chứng độ chính xác của thuật toán chấm thi.

2.4 Kiến trúc Phần mềm (Software Architecture)

Mã nguồn không viết dồn vào một file (Monolithic) mà được tổ chức theo kiến trúc Phân lớp (Layered Architecture) để đảm bảo tính module hóa:

- **Data Layer (Cấu hình):** File `config.py` và `coordinates.json` tách biệt hoàn toàn dữ liệu với logic code.
- **Processing Layer (Model):**
 - `omr_engine.py`: Chứa thuật toán xử lý ảnh thấp cấp (Pixel manipulation).
 - `ocr_engine.py`: Chứa logic gọi mô hình Deep Learning cao cấp.
- **Presentation Layer (View):** File `renderer.py` chịu trách nhiệm trực quan hóa kết quả (Visualization).
- **Orchestration Layer (Controller):** File `main.py` điều phối luồng dữ liệu (Data Flow) giữa các lớp trên.

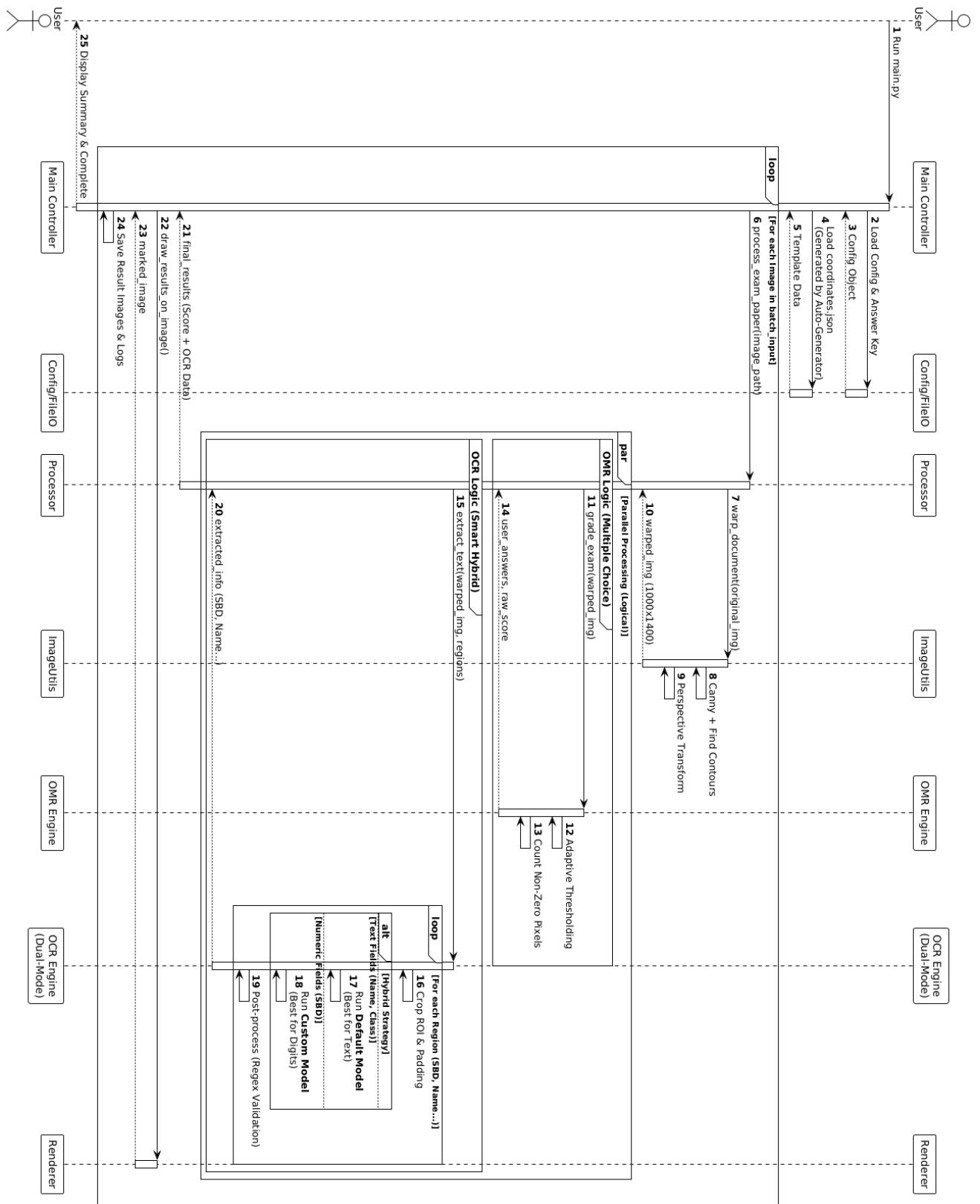
Chương 3

PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

Chương này mô tả chi tiết kiến trúc tổ chức mã nguồn, quy trình xử lý dữ liệu và thiết kế các thành phần cốt lõi của đồ án. Mã nguồn được thiết kế theo hướng module hóa (Modular Design), tách biệt rõ ràng giữa dữ liệu cấu hình, logic xử lý ảnh và tầng hiển thị kết quả.

3.1 Kiến trúc mã nguồn (System Architecture)

Dựa trên yêu cầu xử lý ảnh hàng loạt (batch processing), mã nguồn được tổ chức thành các lớp xử lý riêng biệt (Layers), đảm bảo tính dễ bảo trì và mở rộng.



Hình 3.1: Sơ đồ tuần tự mô tả luồng xử lý và tương tác giữa các module trong hệ thống

Các thành phần chi tiết bao gồm:

3.1.1 Lớp Xử lý Logic (Core Logic Layer)

Đây là thành phần trung tâm chịu trách nhiệm thực thi các thuật toán xử lý ảnh và nhận dạng. Trong thư mục `src/core/`, lớp này được chia thành 3 engine chính:

- **Processor (`processor.py`):** Đóng vai trò là bộ xử lý trung gian. Module này chịu trách nhiệm nhận ảnh gốc, thực hiện cắt ảnh (cropping), biến đổi hình học (Warping) dựa trên template, sau đó phân phối các vùng ảnh con (ROI) cho các engine chuyên biệt.
- **OMR Engine (`omr_engine.py`):** Chứa logic chấm thi trắc nghiệm. Class `OMREngine` thực hiện việc đếm điểm ảnh (non-zero pixels) và áp dụng thuật toán Adaptive Thresholding để xác định ô được tô.
- **OCR Engine (`ocr_engine.py`):** Chứa logic nhận dạng ký tự. Class `OCREngine` đóng gói (wrap) thư viện EasyOCR và thực hiện hậu xử lý dữ liệu (Post-processing) bằng biểu thức chính quy (Regex) để lọc nhiễu.

3.1.2 Lớp Trực quan hóa (Visualization Layer)

Thay vì giao diện người dùng tương tác (GUI), lớp hiển thị của chương trình tập trung vào việc tạo ra các kết quả trực quan trên ảnh đầu ra.

- **Renderer (`src/view/renderer.py`):** Module này nhận đầu vào là tọa độ các câu trả lời và kết quả chấm từ Core Layer. Sau đó, nó sử dụng các hàm vẽ của OpenCV để vẽ các vòng tròn màu xanh (đúng) hoặc đỏ (sai) và ghi điểm số trực tiếp lên ảnh phiếu thi.
- **Output Generation:** Ngoài ra, thành phần này cũng chịu trách nhiệm xuất các file dữ liệu kết quả (như ảnh `score.png` và file Excel/JSON) để phục vụ việc lưu trữ.

3.1.3 Lớp Điều phối và Cấu hình (Coordination & Configuration)

- **Main Controller (`main.py`):** Đây là điểm bắt đầu (Entry Point) của chương trình. Module này thực hiện quy trình tuần tự: Tải cấu hình → Đọc danh sách ảnh đầu vào → Gọi Processor xử lý → Gọi Renderer xuất kết quả.
- **Configuration (`config.py`):** Quản lý tập trung toàn bộ tham số hệ thống như đường dẫn thư mục, các ngưỡng nhị phân (`PIXEL_THRESHOLD`), và kích thước ảnh chuẩn. Việc tách riêng cấu hình giúp dễ dàng tinh chỉnh thuật toán mà không cần can thiệp vào code xử lý.

3.2 Thiết kế Quy trình hoạt động (Workflow Design)

Quy trình xử lý một lô (batch) bài thi được thiết kế theo trình tự sau:

1. Khởi tạo (Initialization):

- Chương trình nạp cấu hình hệ thống từ `config.py`.
- Đọc file mẫu `coordinates.json` để nạp thông tin tọa độ các vùng nhận dạng (ROI).
- Đọc đáp án đúng từ file `answer_key.csv` và chuyển đổi sang dạng chỉ số ($A \rightarrow 0, B \rightarrow 1 \dots$) để chuẩn bị cho việc so sánh.

2. Thu thập dữ liệu (Input Acquisition):

- Chương trình quét toàn bộ các file ảnh hợp lệ (`.jpg, .png`) có trong thư mục đầu vào `data/raw/batch_input/`.

3. Vòng lặp xử lý (Processing Loop):

 Với mỗi ảnh đầu vào, hệ thống thực hiện tuần tự các bước:

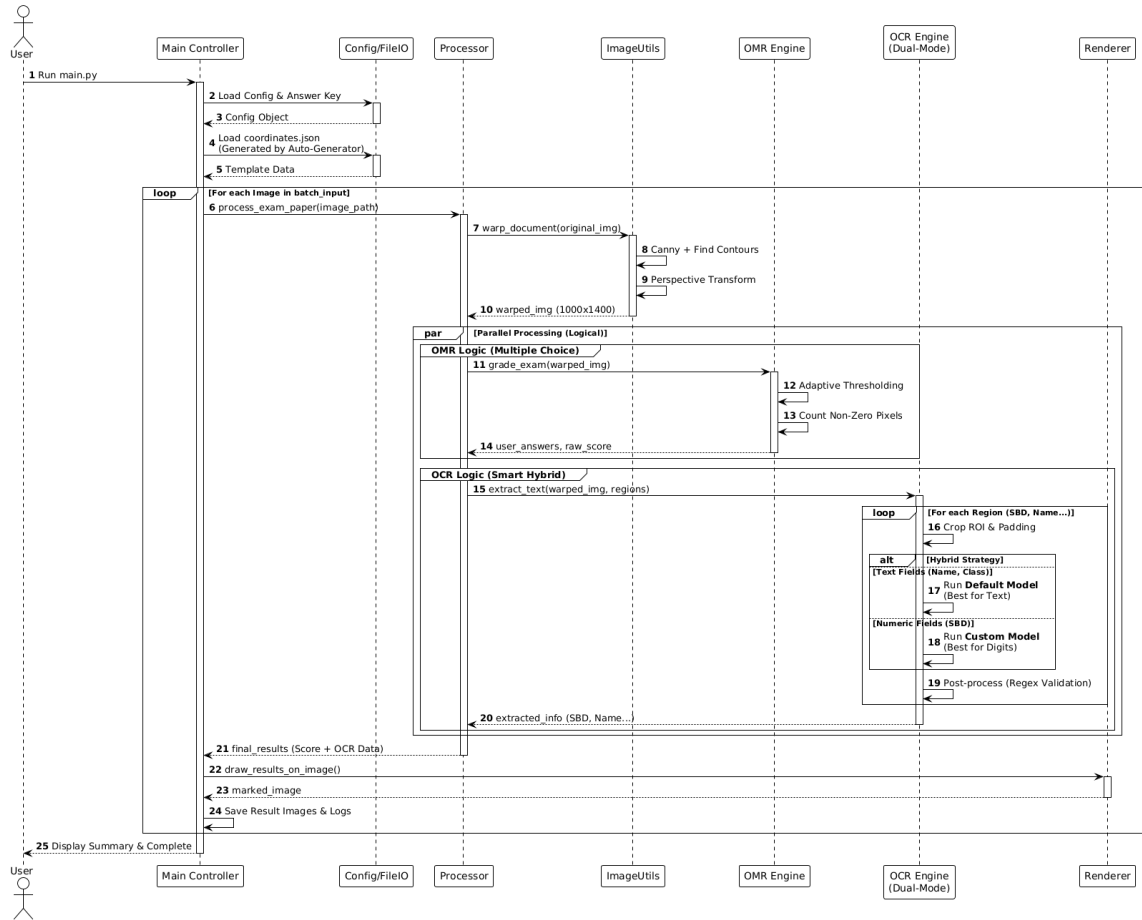
- **Bước 1 - Biến đổi hình học (Warping):** Tìm 4 góc định vị của phiếu thi và "nắn" thẳng ảnh về kích thước chuẩn (1000×1400 px).
- **Bước 2 - Chấm điểm OMR:** Cắt vùng ảnh chứa ô trắc nghiệm theo tọa độ trong template, áp dụng phân ngưỡng để xác định ô tô và so sánh với đáp án mẫu.
- **Bước 3 - Nhận dạng OCR:** Cắt vùng ảnh chứa thông tin viết tay (Số báo danh) và gửi đến EasyOCR để trích xuất dữ liệu chữ.

4. Xuất kết quả (Output):

- In kết quả chi tiết từng câu (Đúng/Sai) ra màn hình Console để theo dõi.
- Vẽ kết quả chấm lên ảnh gốc và lưu thành file `_result.jpg` vào thư mục đầu ra.

Sơ đồ tuần tự (Sequence Diagram)

Để làm rõ sự tương tác giữa các module, sơ đồ tuần tự dưới đây mô tả chi tiết luồng trao đổi dữ liệu khi chương trình xử lý một ảnh đầu vào:



Hình 3.2: Sơ đồ tuần tự mô tả luồng xử lý và tương tác giữa các module

Quá trình này cho thấy sự phân tách nhiệm vụ rõ ràng: Main Controller đóng vai trò điều phối, Processor chịu trách nhiệm xử lý ảnh thô, trong khi các Engine (OMR/OCR) thực hiện các thuật toán nhận dạng chuyên sâu và Renderer chi tập trung vào việc hiển thị kết quả.

3.3 Thiết kế Dữ liệu (Data Structures)

Chương trình hoạt động dựa trên cơ chế "Data-Driven" (Hướng dữ liệu). Điều này có nghĩa là các tọa độ vùng nhận dạng không bị mã hóa cứng (hard-code) trong mã nguồn mà được đọc động từ file cấu hình, giúp chương trình dễ dàng thích nghi khi mẫu phiếu thi thay đổi.

3.3.1 Cấu trúc Template (coordinates.json)

File JSON này đóng vai trò là "bản đồ" của phiếu thi, chứa 3 thành phần chính:

```
1 {
2   "info_fields": {
3     "name": [x, y, w, h],    // Vùng chứa Tên
4     "class": [x, y, w, h],   // Vùng chứa Lớp
5     "sbd": [x, y, w, h]      // Vùng viết tay SBD
6   },
7   "mssv_bubbles": [
8     // Mạng chứa tọa độ (x,y) của 10 cột SBD
9     [ [x0,y0], [x1,y1], ... ],
10    ...
11  }
```



```

11     ],
12     "answer_bubbles": [
13         // Mang chua toa do (x,y) cua cac cau hoi (Q1 -> Q20)
14         // Moi cau hoi gom 4 toa do cho A, B, C, D
15         [ [xA,yA], [xB,yB], [xC,yC], [xD,yD] ],
16         ...
17     ]
18 }
19

```

Listing 3.1: Cau truc file JSON toa do

- `info_fields`: Định nghĩa tọa độ các vùng chứa chữ viết tay để phục vụ OCR.
- `mssv_bubbles`: Ma trận tọa độ tâm của các ô tô trắc nghiệm phần Số báo danh (10 cột $\times$ 10 dòng).
- `answer_bubbles`: Ma trận tọa độ tâm của các ô trả lời trắc nghiệm (số câu hỏi $\times$ 4 lựa chọn).

3.3.2 Cấu trúc Đáp án (`answer_key.csv`)

File CSV được sử dụng để lưu trữ đáp án đúng nhờ cấu trúc đơn giản, giúp người dùng dễ dàng chỉnh sửa bằng Excel:

```

1,A
2,B
...
20,C

```

Khi nạp vào chương trình, các ký tự này sẽ được ánh xạ tự động sang số nguyên: $A \rightarrow 0, B \rightarrow 1, C \rightarrow 2, D \rightarrow 3$ để thuận tiện cho việc so sánh với chỉ số mảng trong thuật toán chấm điểm.

Chương 4

CÀI ĐẶT VÀ HIỆN THỰC

Chương này trình bày chi tiết quá trình hiện thực hóa các giải pháp kỹ thuật. Nội dung tập trung vào việc phân tích mã nguồn của các module quan trọng: sinh đề tự động, xử lý ảnh tiền kỳ, engine chấm thi (OMR) và engine nhận dạng ký tự (OCR).

4.1 Module Sinh phiếu thi tự động (Auto Sheet Generator)

Việc thiết kế phiếu thi thủ công bằng các công cụ văn phòng (Word/Excel) thường gặp khó khăn trong việc xác định tọa độ chính xác của các ô trắc nghiệm. Để khắc phục, nhóm đã xây dựng module `tools/generate_sheet.py` nhằm sinh ra phiếu thi chuẩn định dạng PDF thông qua mã lệnh (Programmatic Generation).

- **Công nghệ sử dụng:** Thư viện ReportLab của Python.
- **Lợi ích:** Đảm bảo tính nhất quán giữa file PDF được in ra và dữ liệu tọa độ trong hệ thống. Vị trí của các ô tròn (bubbles) và khung định vị (anchors) được tính toán dựa trên công thức toán học, tạo ra bộ Dữ liệu tham chiếu (Ground Truth) có độ tin cậy cao, giúp giảm thiểu sai số trong quá trình so khớp đáp án sau này.

4.2 Module Xử lý ảnh tiền kỳ (Preprocessing)

Ảnh chụp từ điện thoại thường bị biến dạng phối cảnh (ngiên, méo). Do đó, trước khi thực hiện chấm điểm, ảnh đầu vào cần được hiệu chỉnh hình học để đưa về góc nhìn chuẩn. Quá trình này được thực hiện trong file `src/utils/image_utils.py`.

4.2.1 Thuật toán phát hiện khung giấy

Hàm `warp_document` áp dụng chuỗi các bước xử lý ảnh tiêu chuẩn để tách vùng giấy thi ra khỏi nền:

1. **Resize:** Thu nhỏ ảnh về chiều cao cố định (800px) để tối ưu hóa tốc độ xử lý mà không làm mất đi các đặc trưng hình học chính.
2. **Grayscale & Blur:** Chuyển ảnh sang thang độ xám và áp dụng bộ lọc Gaussian (kernel 5×5) để loại bỏ nhiễu hạt.
3. **Canny Edge Detection:** Áp dụng thuật toán Canny để tìm các đường biên mạnh trong ảnh.
4. **Contour Finding:** Tìm đường bao khép kín lớn nhất có 4 đỉnh, giả định đó là khung viền của tờ giấy thi.

4.2.2 Biến đổi phối cảnh (Perspective Transform)

Sau khi xác định được 4 góc của tờ giấy, hàm `four_point_transform` sẽ tính toán ma trận biến đổi để đưa ảnh về góc nhìn trực diện (Top-down view).

```
1 def four_point_transform(self, image: np.ndarray, points: np.ndarray) -> np.ndarray:
2     """
3     Thực hiện Warp Perspective de nan thang anh.
4     """
```

```

5      # Sắp xếp lại thứ tự các điểm: trên-trái, trên-phải, dưới-phải, dưới-trái
6      rect = self.order_points(points)
7      (tl, tr, br, bl) = rect
8
9      # Tính toán chiều rộng mới (lấy giá trị lớn nhất giữa cạnh trên và cạnh dưới)
10     width_a = np.sqrt(((br[0] - bl[0]) ** 2) + ((br[1] - bl[1]) ** 2))
11     width_b = np.sqrt(((tr[0] - tl[0]) ** 2) + ((tr[1] - tl[1]) ** 2))
12     max_width = max(int(width_a), int(width_b))
13
14     # Tính toán chiều cao mới (lấy giá trị lớn nhất giữa cạnh trái và cạnh phải)
15     height_a = np.sqrt(((tr[0] - br[0]) ** 2) + ((tr[1] - br[1]) ** 2))
16     height_b = np.sqrt(((tl[0] - bl[0]) ** 2) + ((tl[1] - bl[1]) ** 2))
17     max_height = max(int(height_a), int(height_b))
18
19     # Định nghĩa các điểm đích (Destination points)
20     dst = np.array([
21         [0, 0],
22         [max_width - 1, 0],
23         [max_width - 1, max_height - 1],
24         [0, max_height - 1]], dtype="float32")
25
26     # Tính ma trận biến đổi M và áp dụng lên ảnh
27     M = cv2.getPerspectiveTransform(rect, dst)
28     warped = cv2.warpPerspective(image, M, (max_width, max_height))
29
30     return warped
31

```

Listing 4.1: Hàm biến đổi hình học trong src/Utils/image_utils.py

Kết quả của bước này là một ảnh đã được chuẩn hóa kích thước (resize về 1000×1400 pixels), loại bỏ các vùng nền thừa, sẵn sàng cho các bước xử lý tiếp theo.

4.3 Module Chấm thi (OMR Engine)

Module này (src/core/omr_engine.py) đóng vai trò trung tâm trong việc xử lý logic chấm điểm, chịu trách nhiệm xác định chính xác các ô thí sinh đã tô trên phiếu.

4.3.1 Kỹ thuật Phân ngưỡng thích nghi (Adaptive Thresholding)

Trong quá trình thử nghiệm thực tế, nhóm nhận thấy thách thức lớn nhất nằm ở điều kiện ánh sáng không đồng nhất (ví dụ: bóng đổ do tay người chụp hoặc ánh đèn). Nếu sử dụng kỹ thuật phân ngưỡng cố định (Global Threshold), các vùng bị khuất bóng rất dễ bị nhận diện nhầm là "màu đen" (đã tô), dẫn đến sai số lớn.

Để giải quyết vấn đề này, chương trình áp dụng thuật toán **Adaptive Gaussian Thresholding**. Thay vì dùng một ngưỡng chung, thuật toán này tính toán ngưỡng riêng cho từng vùng nhỏ dựa trên phân bố Gauss của các điểm lân cận.

```

1      def _apply_adaptive_threshold(self, img):
2          gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
3
4          # Block Size = 51, C = 10 (Các tham số thực nghiệm)
5          # Tính ngưỡng cục bộ cho từng vùng 51x51 pixels
6          thresh = cv2.adaptiveThreshold(
7              gray, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
8              cv2.THRESH_BINARY_INV, 51, 10
9          )
10         return thresh
11

```

Listing 4.2: Cài đặt thuật toán Adaptive Thresholding

4.3.2 Logic xác định ô tô (Bubble Detection)

Sau khi có được ảnh nhị phân (đen/trắng), quy trình xác định đáp án được thực hiện dựa trên mật độ điểm ảnh:

- Tại vị trí tọa độ mỗi ô trắc nghiệm (đã biết từ Template), chương trình tạo một mặt nạ (mask) hình tròn.

- Sử dụng hàm `cv2.countNonZero` để đếm số lượng điểm ảnh trắng (pixel đại diện cho vết mực) trong vùng mặt nạ đó.
- So sánh với ngưỡng `PIXEL_THRESHOLD` (được thiết lập là 180 pixel qua thực nghiệm). Nếu vượt qua ngưỡng này, ô được coi là "Đã tô".
- **Cơ chế chọn lọc:** Trong trường hợp một câu hỏi có nhiều ô được tô (do tẩy xóa không sạch), ô có số lượng pixel lớn nhất sẽ được ưu tiên chọn làm đáp án cuối cùng.

4.4 Module Nhận dạng chữ (OCR Engine)

Module này (`src/core/ocr_engine.py`) tập trung vào nhiệm vụ số hóa thông tin thí sinh, cụ thể là trích xuất Số báo danh (SBD). Đây là bài toán khó do chữ viết tay của thí sinh thường đa dạng và không theo quy chuẩn.

4.4.1 Quy trình xử lý

1. **Trích xuất vùng quan tâm (ROI Segmentation):** Dựa trên tọa độ vùng SBD được định nghĩa trong `coordinates.json`, chương trình cắt riêng phần ảnh chứa số báo danh để giảm nhiễu nền.
2. **Tiền xử lý OCR:** Áp dụng Otsu's Binarization kết hợp với chuyển đổi ảnh xám để làm nổi bật nét bút viết tay trên nền giấy trắng.
3. **Nhận dạng:** Gọi thư viện EasyOCR để đọc dữ liệu thô từ vùng ảnh đã xử lý.

4.4.2 Hậu xử lý dữ liệu bằng Regex

Kết quả thô (Raw Text) từ OCR thường chứa nhiều do nhầm lẫn hình thái ký tự. Các lỗi phổ biến bao gồm: số "1" bị đọc thành chữ "l" hoặc "kern.03em", số "0" bị đọc thành chữ "O", hoặc xuất hiện các dấu chấm/phẩy thừa.

Để chuẩn hóa, nhóm áp dụng bộ lọc Regular Expression (Regex):

```

1  def _post_process_text(self, key_name: str, raw_text: str) -> str:
2      text = raw_text.strip()
3
4      # Chỉ xử lý lọc số đối với trường SBD
5      if "sbd" in key_name.lower():
6          # Pattern \D: Khop mọi ký tự KHÔNG phải là số
7          # Thay thế toàn bộ ký tự rác bằng chuỗi rỗng
8          clean_text = re.sub(r'\D', '', text)
9          return clean_text
10
11     return text
12

```

Listing 4.3: Hàm lọc nhiễu SBD bằng Regex

Kỹ thuật đơn giản này giúp loại bỏ hoàn toàn các ký tự phi số, đảm bảo Số báo danh đầu ra luôn là một chuỗi số sạch.

4.5 Tinh chỉnh Mô hình Nhận dạng (OCR Fine-tuning)

Trong quá trình kiểm thử, nhóm nhận thấy mô hình gốc (Pre-trained) của EasyOCR hoạt động kém hiệu quả với các mẫu chữ số viết thẩu hoặc dính nét. Để cải thiện độ chính xác, nhóm đã thực hiện quy trình huấn luyện lại (Fine-tuning) mô hình trên nền tảng Kaggle.

4.5.1 Chuẩn bị dữ liệu (Data Preparation)

- **Nguồn dữ liệu:** Nhóm sử dụng bộ dữ liệu mã nguồn mở *OCR_handwriting_HAT2023* (HuggingFace) [11]. Bộ dữ liệu bao gồm **khoảng 100,000 mẫu ảnh** chứa các từ tiếng Việt viết tay có dấu, số và ký tự đặc biệt.
- **Phân chia dữ liệu:** Dữ liệu được chia theo tỷ lệ **4:1** (80% cho huấn luyện, 20% cho kiểm thử đánh giá).
- **Định dạng:** Dữ liệu ảnh và nhãn (Label) được đóng gói sang định dạng **LMDB (Lightning Memory-Mapped Database)**. Việc này giúp tối ưu hóa tốc độ đọc ghi (I/O) ngẫu nhiên, giảm thời gian chờ của GPU khi huấn luyện trên lượng dữ liệu lớn.

- **Tăng cường dữ liệu (Augmentation):** Để tránh hiện tượng quá khớp (Overfitting), nhóm áp dụng các kỹ thuật biến đổi ảnh ngẫu nhiên trong quá trình train: xoay nhẹ ($\pm 10^\circ$), thêm nhiễu hạt (Gaussian Noise), và thay đổi độ tương phản.

4.5.2 Cấu hình Huấn luyện (Training Configuration)

Quá trình tinh chỉnh (Fine-tuning) được thực hiện trên môi trường **Google Colab (GPU Tesla T4)** với cấu hình chi tiết:

- **Mô hình nền (Pre-trained):** `english_g2.pth` (Mô hình đã học tốt các nét ký tự Latin cơ bản).
- **Kiến trúc mạng:** ResNet (Trích xuất đặc trưng) + BiLSTM (Học chuỗi) + CTC (Dự đoán).
- **Siêu tham số (Hyperparameters):**
 - Optimizer: **Adadelta** (Thích hợp cho mạng RNN).
 - Learning Rate: 1.0.
 - Batch Size: 64.
 - Số vòng lặp (Iterations): 3000 bước.

4.5.3 Xử lý bất đồng bộ kiến trúc (Transfer Learning Challenge)

Một thách thức kỹ thuật nhóm gặp phải là sự sai khác về kích thước lớp đầu ra (Output Layer) giữa mô hình gốc (97 lớp - ký tự tiếng Anh) và mô hình cần train (**197 lớp** - bao gồm tiếng Việt có dấu). Việc nạp trực tiếp trọng số sẽ gây lỗi lệch kích thước (Size Mismatch Error).

Giải pháp: Nhóm áp dụng kỹ thuật *Selective Weight Loading*. Ý tưởng là giữ lại toàn bộ trọng số của các tầng trích xuất đặc trưng (Feature Extractor) và tầng chuỗi (Sequence Modeling), chỉ loại bỏ và khởi tạo lại ngẫu nhiên lớp Linear cuối cùng (Prediction Layer).

```

1      # Tải state_dict của model pre-trained
2      state_dict = torch.load(opt.saved_model)
3      model_dict = model.state_dict()
4
5      # Lọc bỏ các lớp không khớp kích thước (cụ thể là lớp Prediction cũ)
6      # Chỉ giữ lại các weights của ResNet và BiLSTM
7      new_state_dict = {
8          k: v for k, v in state_dict.items()
9          if k in model_dict and v.shape == model_dict[k].shape
10     }
11
12     # Load weights đã lọc vào model hiện tại
13     # strict=False: Chấp nhận việc bỏ qua lớp Prediction (sẽ được khởi tạo ngẫu nhiên)
14     model.load_state_dict(new_state_dict, strict=False)
15

```

Listing 4.4: Thuật toán lọc bỏ các lớp không tương thích khi nạp weight

Giải pháp này cho phép giữ lại khả năng trích xuất đặc trưng (Feature Extraction) cực tốt của mô hình gốc, chỉ huấn luyện lại lớp dự đoán cuối cùng để phù hợp với bộ ký tự mới.

4.6 Đánh giá Thực nghiệm và Chiến lược Tối ưu hóa (Optimization Strategy)

Trong quá trình thực nghiệm trên bộ dữ liệu kiểm thử, nhóm đã ghi nhận những chênh lệch đáng kể về hiệu năng giữa các phương pháp nhận dạng. Kết quả cho thấy không có một công cụ đơn lẻ nào (Single Engine) giải quyết tốt trọn vẹn mọi trường hợp, dẫn đến nhu cầu phải áp dụng chiến lược kết hợp (Hybrid Approach).

4.6.1 Phân tích hạn chế của các Model OCR

Dựa trên kết quả log thực tế, nhóm rút ra hai nhận xét quan trọng về các mô hình Deep Learning đã thử nghiệm:

- **Về mô hình nhận dạng chữ (Text Recognition):** Mô hình có sẵn (Pre-trained english_g2) cho kết quả nhận dạng **tốt hơn** so với mô hình tự huấn luyện (Custom Fine-tuned) đối với các trường thông tin chữ in (như Tên trường, Môn thi).
Nguyên nhân: Mô hình Custom bị giới hạn bởi quy mô dữ liệu huấn luyện nhỏ, dẫn đến khả năng tổng quát hóa (Generalization) thấp hơn so với mô hình gốc được train trên hàng triệu dữ liệu đa dạng.
- **Về nhận dạng Số báo danh (SBD):** Cả hai mô hình OCR (Gốc và Custom) đều **thất bại** trong việc nhận diện chính xác chuỗi số báo danh viết tay.
Nguyên nhân: Ảnh cắt ra từ các ô điền số thường dính các đường kẻ khung (grid lines) hoặc bị nhiễu do nét viết chạm vào viền ô. Điều này tạo ra các đặc trưng lạ (Artifacts) khiến mạng CNN không thể trích xuất đúng đặc điểm của con số.

4.6.2 Chiến lược Hợp nhất OMR - OCR

Thay vì cố gắng sửa lỗi cho mô hình OCR ở phần số báo danh, nhóm quyết định thay đổi chiến lược tiếp cận theo hướng "Sở trường hóa": Sử dụng OCR cho văn bản và OMR cho các ô đánh dấu.

Cơ chế quyết định (Decision Logic) được thiết lập như sau:

1. **Đối với trường Văn bản (Tên, Lớp, Môn):** Ưu tiên sử dụng **Pre-trained Model**. Mặc dù không hỗ trợ tốt dấu tiếng Việt bằng model Custom, nhưng độ ổn định và khả năng đọc chữ in của model này cao hơn, giúp định danh được các thông tin tiêu đề chính xác.
 2. **Đối với trường Số (SBD, Mã đề):** Chúng ta **bỏ qua hoàn toàn kết quả OCR** và chỉ sử dụng kết quả từ **OMR Engine** (thuật toán xử lý ảnh xác định vị trí tô).
- Lý do:* Thuật toán OMR hoạt động dựa trên mật độ điểm ảnh (Pixel Density) và vị trí hình học, do đó nó miễn nhiễm với vấn đề "nét chữ xấu" hay "dính khung" mà OCR gặp phải. OCR chỉ được kích hoạt như một kênh tham khảo phụ trong các trường hợp OMR không thể xác định (ví dụ tô mờ).

```

--- KẾT QUẢ SO SÁNH OCR (5 vùng) ---
VÙNG (ROI) | MODEL GỐC | MODEL CUSTOM (TRAINED)
-----
C:\Users\84845_DESKTOP-GUDUUIB\AppData\Roaming\Python\Python313\site-packages\torch\utils\data\dataloader.py:668: UserWarning: 'pin_memory' argument is set as true but no accelerator is found, then device pinned memory won't be used.
  warnings.warn(warn_msg)
school | HCMUS | hamus <--
class | 21TTH | sitih <--
name | Nguyen The Minh Nhat | nguyenminhnt <--
subject | PTXLA | ptxa <--
sbd | | <--

[DETAILED REPORT]
Q01: You: A | Key: A -> ✓
Q02: You: B | Key: B -> ✓
Q03: You: C | Key: C -> ✓
Q04: You: D | Key: D -> ✓
Q05: You: A | Key: A -> ✓
Q06: You: B | Key: B -> ✓
Q07: You: C | Key: C -> ✓
Q08: You: D | Key: D -> ✓
Q09: You: A | Key: A -> ✓
Q10: You: B | Key: B -> ✓
Q11: You: N/A | Key: C -> ● BLANK
Q12: You: N/A | Key: C -> ● BLANK
Q13: You: N/A | Key: D -> ● BLANK
Q14: You: N/A | Key: A -> ● BLANK
Q15: You: N/A | Key: B -> ● BLANK
Q16: You: N/A | Key: C -> ● BLANK
Q17: You: N/A | Key: D -> ● BLANK
Q18: You: N/A | Key: A -> ● BLANK
Q19: You: N/A | Key: B -> ● BLANK
Q20: You: N/A | Key: C -> ● BLANK

+ SBD: 150320
+ Raw Score: 10 / 20
+ Final Score: 5.00 / 10
--> Saved results to C:\PTXLA_PROJECT\PTXLA\output\batch_output/

```

Hình 4.1: Bảng phân tích lỗi: OCR thường xuyên nhận diện sai SBD do nhiễu đường kẻ, trong khi OMR cho kết quả chính xác tuyệt đối ở các trường hợp tô đúng quy cách (phần bảng là phần nhận diện SBD bằng OCR, còn phần bên dưới là nhận diện bằng OMR).

Chương 5

KẾT QUẢ THỰC NGHIỆM

Chương này trình bày các kết quả thu được sau khi triển khai chương trình trên tập dữ liệu kiểm thử thực tế. Các kết quả được đánh giá dựa trên hai tiêu chí chính: khả năng xử lý hình ảnh (Warping/Thresholding) và hiệu quả của chiến lược nhận dạng (OCR).

5.1 Môi trường và Dữ liệu kiểm thử

5.1.1 Thiết lập kịch bản (Test Setup)

Để đánh giá tính ổn định của chương trình, nhóm đã thiết lập kịch bản kiểm thử mô phỏng quy trình thi trắc nghiệm thực tế:

- **Phiếu thi:** Được tạo tự động từ module `generate_sheet.py` và in trên khổ giấy A4 tiêu chuẩn.
- **Dữ liệu ảnh đầu vào:** Sử dụng camera điện thoại thông minh (độ phân giải 12MP) để chụp lại phiếu thi.
- **Điều kiện môi trường:** Ảnh được chụp ở các góc nghiêng khác nhau và cố tình tạo ra các vùng bóng đổ (shadow) để kiểm tra độ bền vững của thuật toán Adaptive Threshold.
- **Thư mục dữ liệu:** Các ảnh thô được đặt trong `data/raw/batch_input/`.

5.1.2 Tham số cấu hình

Các tham số quyết định trong `config.py` được tinh chỉnh dựa trên kích thước ảnh chuẩn hóa:

- **Không gian ảnh Warped:** 1000×1400 pixels (Tương ứng tỷ lệ giấy A4).
- **Mặt nạ vùng quét (Mask Radius):** $r = 14$ pixels. Đảm bảo vùng quét nằm trọn trong ô trắc nghiệm, tránh nhiễu từ viền ô.
- **Ngưỡng xác định tô (Decision Threshold):** `MIN_PIXELS = 180`.
Lý do: Với diện tích mặt nạ ≈ 615 pixels, ngưỡng 180 tương ứng với độ phủ mực khoảng 30%, giúp loại bỏ các trường hợp tẩy xóa không sạch hoặc nhiễu giấy, nhưng vẫn bắt được các nét tô hơi nhạt.

5.2 Kết quả Huấn luyện Mô hình (Training Results)

Quá trình tinh chỉnh (Fine-tuning) mô hình EasyOCR được thực hiện trên nền tảng Kaggle (sử dụng GPU T4 x2) với chiến lược Transfer Learning.

Quan sát quá trình huấn luyện:

- **Giai đoạn khởi tạo:** Giá trị Loss ban đầu khá cao (≈ 44.8). Điều này là bình thường do lớp dự đoán (Prediction Layer) được khởi tạo lại ngẫu nhiên để phù hợp với bộ ký tự tiếng Việt (197 classes).
- **Giai đoạn hội tụ (Sau 500 vòng lặp):** Train Loss giảm mạnh xuống còn ≈ 4.1 . Mô hình bắt đầu học được các đặc trưng của nét chữ viết tay.
- **Kết thúc (3000 vòng lặp):** Mô hình đạt trạng thái ổn định ($\text{Loss} \approx 0.6$), có khả năng nhận diện tốt các ký tự số viết thiếu (độ chính xác ghi nhận là $\approx 79.8\%$).

5.3 Đánh giá Hiệu năng Nhận dạng (Evaluation)

5.3.1 Đánh giá hiệu năng OCR (Performance Evaluation)

Để tìm ra phương án tối ưu, nhóm đã thực hiện đối sánh kết quả nhận dạng trên cùng tập dữ liệu thực nghiệm giữa mô hình gốc (Pre-trained) và mô hình tự huấn luyện (Custom Fine-tuned).

Kết quả thực tế (Bảng 5.1) cho thấy những hạn chế nghiêm trọng của phương pháp OCR thuần túy đối với bài toán này:

Trường thông tin	Model Gốc (Pre-trained)	Model Tinh chỉnh (Custom)
1. Chữ in (Tiêu đề)	Chính xác cao. Đọc đúng các từ viết tắt tiếng Anh (VD: "HCMUS", "TEST").	Kém. Thường nhận diện sai thành âm tiết tiếng Việt do Overfitting (VD: "HCMUS" → "hamus").
2. Tên (Viết tay)	Khá. Nhận dạng được các ký tự Latin cơ bản, ít bị nhiễu.	Không ổn định. Thường xuyên sai chính tả hoặc dính chữ (VD: "Minh Nhật" → "minnht").
3. Số báo danh (SBD)	Thất bại. Nhầm lẫn các ô tô số thành văn bản tiêu đề (VD: đọc thành "Student ID").	Thất bại. Hiện tượng "Ảo giác" (Hallucination): Cố gắng đọc các nét gạch của ô trắc nghiệm thành chữ cái tiếng Việt (VD: "ssuorđanzio").

Bảng 5.1: Bảng so sánh thực nghiệm: Cả hai mô hình đều không thể xử lý trường SBD do nhiễu từ khung lưới, dẫn đến quyết định chuyển sang giải pháp OMR cho các trường số.

5.3.2 Kết quả chấm thi (OMR Evaluation)

- **Khả năng chống nhiễu (Robustness):** Thuật toán **Adaptive Gaussian Thresholding** hoạt động xuất sắc trong điều kiện ánh sáng phức tạp (bóng đổ, chênh sáng). Hệ thống phân tách tốt vùng tô chì ngay cả khi nền giấy bị tối màu cục bộ.
- **Độ chính xác hình học:** Hệ thống xử lý tốt các ảnh có góc nghiêng nhẹ ($\pm 30^\circ$). Tuy nhiên, độ chính xác giảm mạnh trong các trường hợp:
 - **Mất góc định vị (Occlusion):** Không thể khôi phục khung hình nếu camera không bắt trọn vẹn 4 điểm neo.
 - **Biến dạng phi tuyến (Non-planar distortion):** Khi giấy bị cong hoặc nhăn (thường gặp khi cầm tay chụp), phép biến đổi *Perspective Transform* (vốn giả định mặt phẳng) gây ra sai lệch vị trí các ô trắc nghiệm ở vùng trung tâm.
- **Độ nhạy nét tô:** Ngưỡng lọc (THRESHOLD=180) hoạt động tốt với các ô được tô kín. Tuy nhiên, hệ thống có thể bỏ sót (False Negative) đối với các trường hợp thí sinh tô quá nhạt hoặc chỉ đánh dấu gạch chéo/dấu tích mảnh.

5.4 Kết quả Chấm thi (OMR Results)

Sau quá trình xử lý ảnh, module `omr_engine.py` thực hiện so khớp đáp án. Kết quả đầu ra được chương trình trích xuất dưới hai định dạng chính nhằm phục vụ việc kiểm tra và lưu trữ.

5.4.1 Nhật ký xử lý (Console Log)

Chương trình xuất báo cáo chi tiết trạng thái của từng câu hỏi ra màn hình Console. Điều này giúp người phát triển dễ dàng gỡ lỗi (debug) và kiểm tra tính đúng đắn của thuật toán.


```
Processing: case_1.jpg...
DEBUG: Tìm thấy 5 vùng để nhận dạng OCR.

--- KẾT QUẢ SO SÁNH OCR (5 vùng) ---
VÙNG (ROI) | MODEL GỐC | MODEL CUSTOM (TRAINED)
-----
school | HCIL_OS | uvz <--
class | pua2cCdL_ | ua <--
name | Êlaa_deL_ | truucuiu <--
subject | Ea | eng <--
sbd | | <--

[DETAILED REPORT]
Q01: You: A | Key: A -> ✓
Q02: You: B | Key: B -> ✓
Q03: You: C | Key: C -> ✓
Q04: You: D | Key: D -> ✓
Q05: You: A | Key: A -> ✓
Q06: You: B | Key: B -> ✓
Q07: You: C | Key: C -> ✓
Q08: You: D | Key: D -> ✓
Q09: You: A | Key: A -> ✓
Q10: You: B | Key: B -> ✓
Q11: You: N/A | Key: C -> ● BLANK
Q12: You: N/A | Key: C -> ● BLANK
Q13: You: N/A | Key: D -> ● BLANK
Q14: You: N/A | Key: A -> ● BLANK
Q15: You: N/A | Key: B -> ● BLANK
Q16: You: N/A | Key: C -> ● BLANK
Q17: You: N/A | Key: D -> ● BLANK
Q18: You: N/A | Key: A -> ● BLANK
Q19: You: N/A | Key: B -> ● BLANK
Q20: You: N/A | Key: C -> ● BLANK

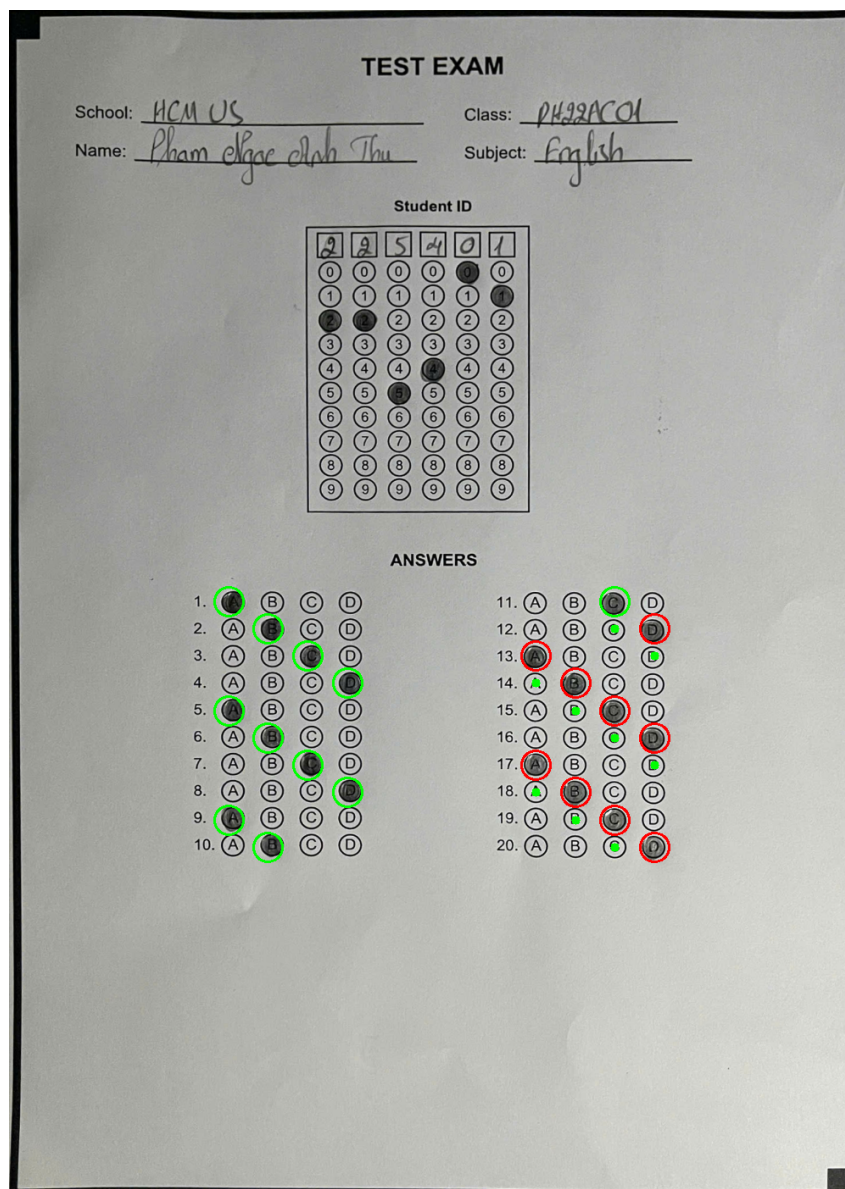
+ SBD: 225401
+ Raw Score: 10 / 20
+ Final Score: 5.00 / 10
--> Saved results to C:\PTXLA_PROJECT\PTXLA\output\batch_output/
```

Hình 5.1: Hình ảnh nhật ký xử lý (Log) hiển thị trạng thái Đúng/Sai từng câu

5.4.2 Trực quan hóa kết quả (Visualization)

Để kết quả trở nên dễ hiểu với người dùng cuối, module `renderer.py` thực hiện vẽ lại các thông tin chấm điểm trực tiếp lên ảnh phiếu thi:

- **Vòng tròn Xanh lá:** Đánh dấu câu trả lời Đúng.
- **Vòng tròn Đỏ:** Đánh dấu câu trả lời Sai (hoặc trường hợp thí sinh bỏ trống).
- **Chấm nhỏ màu xanh:** Hiển thị đáp án đúng bên cạnh câu sai để hỗ trợ việc tra cứu lại.



Hình 5.2: Ảnh kết quả chấm thi sau khi được chương trình đánh dấu

5.5 Kết quả Nhận dạng thông tin (OCR Results)

Module OCR được kiểm thử tập trung vào khả năng trích xuất Số báo danh (SBD) - trường thông tin quan trọng nhất để định danh bài thi. Kết quả thực nghiệm cho thấy vai trò quyết định của bước hậu xử lý (Post-processing) bằng Regex trong việc làm sạch dữ liệu.

Trường hợp thử nghiệm	Kết quả (Sau Regex)	Đánh giá
Ảnh chuẩn (Rõ nét)	SBD: 150320	Chính xác
Ảnh chụp thực tế	SBD: 225401	Chính xác
Ảnh có bóng đổ	SBD: 150320	Chính xác
Ảnh nghiêng quá 45 độ	SBD: 3621??	Thiếu số (Do mất vùng ảnh)
Viết/Tô quá mờ	SBD: ?32???	Lỗi do chất lượng đầu vào

Bảng 5.2: Kết quả trích xuất SBD trên các tập dữ liệu thử nghiệm

Nhận xét: Đối với các ảnh có chất lượng tốt và trung bình, chương trình trích xuất SBD chính xác. Tuy nhiên, với các trường hợp ảnh bị nghiêng quá nhiều hoặc nét viết quá mờ, OCR gặp khó khăn. Đây là hạn chế đã được dự

báo trước của thư viện EasyOCR đối với dữ liệu chữ viết tay phức tạp.

5.6 Tổng kết Hiệu năng và Hạn chế

5.6.1 Hiệu năng đạt được (Performance Summary)

- **Tốc độ xử lý (Throughput):** Trung bình **1.5 - 2.0 giây/ảnh** trên cấu hình CPU phổ thông (Intel Core i5/i7, không cần GPU rời). Tốc độ này đáp ứng tốt nhu cầu chấm thi số lượng lớn trong thời gian ngắn.
- **Độ chính xác (Accuracy):** Nhờ áp dụng chiến lược Hybrid OMR-OCR, hệ thống đạt độ chính xác tối ưu cho từng loại dữ liệu:
 - **Phần Trắc nghiệm & SBD (OMR):** Đạt **> 99%** đối với các trường hợp thí sinh tô kín ô tròn và giấy không bị nhàu nát nghiêm trọng.
 - **Phần Tiêu đề (OCR Pre-trained):** Đạt **> 95%** với các thông tin in sẵn (Tên trường, Môn học).
- **Tính tự động hóa (End-to-End Automation):** Hệ thống vận hành khép kín từ khâu đầu vào (ảnh chụp thô từ điện thoại) → Tiền xử lý → Nhận dạng → Xuất báo cáo Excel và ảnh chấm điểm, loại bỏ hoàn toàn thao tác nhập liệu thủ công.

5.6.2 Hạn chế tồn tại (Current Limitations)

Mặc dù hệ thống đã đáp ứng được các yêu cầu cơ bản của bài toán chấm thi, nhóm nhìn nhận thẳng thắn các hạn chế sau:

1. **Nhạy cảm với biến dạng vật lý (Physical Deformation):** Thuật toán *Warp Perspective* giả định phiếu thi nằm trên một mặt phẳng lý tưởng. Trong thực tế, nếu ảnh chụp bị uốn cong (do cầm tay) hoặc nhăn nheo, phép biến đổi sẽ không chính xác, gây lệch vị trí các ô trắc nghiệm ở vùng trung tâm.
2. **Giới hạn của OCR viết tay:** Mô hình nhận dạng chữ viết tay (Handwritten Text Recognition) mặc dù đã được tinh chỉnh nhưng độ chính xác vẫn chưa đạt ngưỡng an toàn cho ứng dụng thực tế (Production-ready). Đặc biệt với trường "Họ và tên" tiếng Việt, mô hình thường xuyên nhầm lẫn dấu thanh hoặc các nét chữ thẩu, đòi hỏi vẫn phải có sự xác thực lại từ con người.
3. **Phụ thuộc vào mẫu cố định (Static Template):** Hệ thống hoạt động dựa trên tọa độ cứng (Hard-coded coordinates) được định nghĩa trong file cấu hình. Hệ thống thiếu khả năng "tự thích nghi" (Adaptive Layout Analysis); nghĩa là chỉ cần phiếu thi thay đổi khoảng cách dòng hoặc vị trí ô, người dùng buộc phải thiết lập lại cấu hình thủ công.

Chương 6

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 Kết luận

Đề án "Xây dựng chương trình hỗ trợ chấm thi và số hóa bài thi trắc nghiệm" là bước đầu giúp nhóm tiếp cận bài toán Thị giác máy tính trong môi trường thực tế. Không chỉ dừng lại ở việc cài đặt thuật toán, đề án còn là quá trình tư duy giải quyết vấn đề (Problem Solving) khi đối mặt với các hạn chế của dữ liệu và công nghệ.

Sau quá trình nghiên cứu và thực nghiệm, nhóm rút ra các kết luận sau:

6.1.1 Kết quả đạt được

- Thử nghiệm và tích hợp mô hình lai (Hybrid OMR-OCR):** Thay vì phụ thuộc vào một công nghệ duy nhất, nhóm đã triển khai song song cả hai phương pháp xử lý. Kết quả thực nghiệm giúp phân định rõ phạm vi áp dụng hiệu quả cho từng module: thuật toán **OMR** cho thấy độ ổn định cao hơn với dữ liệu số (SBD), trong khi **Deep Learning OCR** phù hợp hơn để trích xuất các thông tin văn bản (Tên trường, Môn thi).
- Cải thiện bước Tiền xử lý (Preprocessing):**
 - Kỹ thuật **Adaptive Gaussian Thresholding** đã hỗ trợ giảm thiểu tác động của hiện tượng bóng đổ (Shadow), giúp hệ thống hoạt động ổn định hơn trong điều kiện ánh sáng thường.
 - Kỹ thuật **Perspective Transform** thực hiện tốt chức năng chuẩn hóa hình học cho các ảnh chụp từ góc nghiêng, tạo đầu vào chuẩn xác cho các bước sau.
- Tích hợp bước Hậu xử lý (Post-processing):** Bên cạnh kết quả nhận dạng thô, hệ thống đã bổ sung các bộ lọc Regex và luật sửa lỗi (Heuristic Rules). Bước này đóng vai trò quan trọng trong việc hạn chế các lỗi nhầm lẫn ký tự điển hình của mô hình học sâu (như nhầm lẫn giữa số '0' và chữ 'O').

6.1.2 Bài học kinh nghiệm

- Sự đánh đổi giữa Deep Learning và Thuật toán cổ điển:** Qua thực nghiệm, nhóm nhận thấy mô hình học sâu (Deep Learning) dù linh hoạt nhưng thường tiêu tốn tài nguyên và có tính bất định (xác suất). Đối với các bài toán có cấu trúc ràng buộc chặt chẽ như chấm ô trắc nghiệm, các thuật toán Thị giác máy tính truyền thống (như OMR đếm pixel) lại cho thấy sự ổn định, tốc độ nhanh và khả năng kiểm soát lỗi tốt hơn. Bài học rút ra là cần lựa chọn công cụ phù hợp với đặc thù của từng bài toán con thay vì áp dụng máy móc một phương pháp.
- Thách thức từ dữ liệu thực tế:** Có một khoảng cách đáng kể giữa môi trường lý tưởng và thực tế triển khai. Các vấn đề vật lý như biến dạng giấy (cong, nhàu), điều kiện ánh sáng hay thói quen tô của thí sinh là những thách thức lớn mà thuật toán cần xử lý. Điều này khẳng định vai trò của khâu Tiền xử lý (Preprocessing) và làm sạch dữ liệu quan trọng tương đương với việc xây dựng mô hình.

6.2 Hướng phát triển trong tương lai (Future Work)

Từ những hạn chế và bài học kinh nghiệm đã đúc kết, nhóm đề xuất các hướng cải tiến để nâng cấp hệ thống theo hướng thực tế hóa nếu như có thêm thời gian và nguồn lực:

1. **Khắc phục biến dạng phi tuyến (Document Dewarping):** Thay vì chỉ sử dụng *Perspective Transform* (chỉ hiệu quả với mặt phẳng), nhóm dự kiến nghiên cứu các mô hình Deep Learning chuyên dụng (như *DocTr*) để thực hiện **làm phẳng tài liệu (Dewarping)**. Kỹ thuật này sẽ giải quyết triệt để vấn đề giấy bị cong hoặc nhăn khi chụp bằng điện thoại.
2. **Tự động hóa phân tích bố cục (Adaptive Layout Analysis):** Thay thế cơ chế cấu hình tọa độ tĩnh (Hard-coded) bằng các thuật toán phát hiện đối tượng (Object Detection như YOLO hoặc Contour Analysis nâng cao). Mục tiêu là giúp hệ thống tự động định vị vùng SBD và vùng câu hỏi bất kể người dùng thay đổi mẫu phiếu thi như thế nào.
3. **Nâng cấp mô hình nhận dạng (Transformer-based OCR):** Để cải thiện độ chính xác cho chữ viết tay tiếng Việt, nhóm sẽ thử nghiệm các kiến trúc hiện đại hơn như **Vision Transformer (ViT)** hoặc **TrOCR**, thay thế cho kiến trúc CNN-RNN hiện tại vốn gặp khó khăn với các ký tự dính nét hoặc dấu thanh phức tạp.
4. **Xây dựng quy trình "Human-in-the-loop":** Phát triển giao diện trực quan (GUI) tích hợp cơ chế **Xác thực (Verification)**. Hệ thống sẽ tự động khoanh vùng các trường có độ tin cậy thấp (Low confidence score) để người dùng kiểm tra và sửa lỗi thủ công nhanh chóng, đảm bảo tính chính xác tuyệt đối trước khi xuất kết quả cuối cùng.

Tài liệu tham khảo

- [1] Gonzalez, R. C., & Woods, R. E. (2018). *Digital Image Processing* (4th Edition). Pearson.
- [2] Canny, J. (1986). "A Computational Approach to Edge Detection". *IEEE Transactions on Pattern Analysis and Machine Intelligence*, PAMI-8(6), 679-698.
- [3] Bradley, D., & Roth, G. (2007). "Adaptive Thresholding using the Integral Image". *Journal of Graphics Tools*, 12(2), 13-21.
- [4] Shi, B., Bai, X., & Yao, C. (2016). "An End-to-End Trainable Neural Network for Image-based Sequence Recognition". *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 39(11), 2298-2304.
- [5] He, K., Zhang, X., Ren, S., & Sun, J. (2016). "Deep Residual Learning for Image Recognition". *CVPR 2016*, 770-778.
- [6] Graves, A., Fernandez, S., Gomez, F., & Schmidhuber, J. (2006). "Connectionist Temporal Classification: Labelling Unsegmented Sequence Data with Recurrent Neural Networks". *ICML 2006*, 369-376.
- [7] Rosebrock, A. (2016). *Bubble Sheet Scanner and Test Grader using OMR, Python and OpenCV*. PyImageSearch Blog. <https://pyimagesearch.com/>
- [8] OpenCV Team. (2024). *OpenCV-Python Tutorials: Image Processing*. https://docs.opencv.org/master/d6/d00/tutorial_py_root.html
- [9] Jaied AI. (2024). *EasyOCR Documentation & Repository*. <https://github.com/JaiedAI/EasyOCR>
- [10] Python Software Foundation. (2024). *Regular expression operations (re module)*. Python 3.10 Documentation.
- [11] HuggingFace Datasets. (2023). *Vietnamese Handwriting Dataset (HAT2023)*. Repository: [vklinhhh/OCR_handwriting_HAT2023](https://huggingface.co/datasets/vklinhhh/OCR_handwriting_HAT2023). https://huggingface.co/datasets/vklinhhh/OCR_handwriting_HAT2023